

Módulo de Preguntas de Lógica y Diseño Arquitectónico

Actividad para Aprendices SENA

Parte 1 — Arquitectura

Preguntas

1. ¿Por qué es importante separar rutas y controladores?

2. ¿Qué problemas existirían si toda la lógica estuviera en app.js?

3. ¿Qué responsabilidad tiene un controlador?

4. ¿Qué responsabilidad tiene una ruta?

5. ¿Por qué se usa arquitectura por capas?

Parte 2 — Base de Datos

6. ¿Por qué el correo debe ser UNIQUE?

7. ¿Por qué no se deben guardar passwords sin hash?

8. ¿Qué ventaja tiene usar SQL parametrizado?

Ejemplo:



WHERE correo = ?

Parte 3 — Request y Response

Analizar Request

```
{  
  "nombre": "Carlos",  
  "correo": "carlos@gmail.com",  
  "password": "123456"  
}
```

Preguntas

- ¿Qué datos viajan desde el cliente?
 - ¿Qué datos son sensibles?
 - ¿Qué validaciones harías?
-

Analizar Response

```
{  
  "success": true,  
  "message": "Usuario registrado",  
  "data": {  
    "id": 1  
  },  
  "error": null  
}
```

Preguntas

- ¿Por qué es importante unificar respuestas?



- ¿Qué ventajas tiene para frontend?
 - ¿Qué significa success?
-

Parte 4 — Diseño de API

Diseñar endpoint

Crear endpoint:

PUT /api/users/:id

Preguntas

- ¿Qué hace?
 - ¿Qué método HTTP usa?
 - ¿Qué datos recibe?
 - ¿Qué response debería retornar?
-

Parte 5 — Lógica Backend

Analizar flujo login

Cliente

↓

Ruta

↓

Controlador

↓

SQL

↓

Validación password

↓



Response

Explicar:

- ¿Qué sucede en cada paso?
 - ¿Dónde ocurre la lógica?
 - ¿Dónde ocurre la conexión MySQL?
-

6. Ejercicio Integrador SENA

Objetivo

El aprendiz deberá construir:

- ✓ CRUD completo
 - ✓ Login funcional
 - ✓ Arquitectura organizada
 - ✓ SQL directo
 - ✓ Respuestas unificadas
-

Requerimientos

Obligatorios

- Registro
 - Login
 - Listado
 - Eliminación
 - Actualización usuario
-

Reglas Técnicas

- ✓ Express
- ✓ MySQL



- ✓ SQL parametrizado
- ✓ Variables entorno
- ✓ Arquitectura capas
- ✓ Hash password

Retos Adicionales

Nivel Intermedio

- Validar campos vacíos
- Validar correo
- Verificar longitud password

Lineamientos de Repositorios y Entrega del Proyecto

Objetivo

El aprendiz deberá entregar:

Repositorio	Contenido
Repositorio 1	Proyecto base explicado en clase
Repositorio 2	Solución del reto propuesto

1. Estructura de Entrega

Repositorio 1 — Proyecto Base

Debe contener:



- ✓ API funcional presentada en clase
- ✓ Arquitectura por capas
- ✓ CRUD básico usuarios
- ✓ Login
- ✓ SQL directo
- ✓ Variables de entorno
- ✓ README.md documentado

Nombre sugerido:

api-node-mysql-base

Repositorio 2 — Proyecto Reto

Debe contener:

- ✓ Solución propia del aprendiz
- ✓ Mejoras implementadas
- ✓ Nuevos endpoints
- ✓ Validaciones adicionales
- ✓ Arquitectura organizada
- ✓ README técnico

Nombre sugerido:

api-node-mysql-reto

2. ¿Qué NO se debe subir?

✗ Nunca subir:

node_modules/

.env

¿Por qué no se sube node_modules?

Porque:



- pesa demasiado,
- GitHub se vuelve lento,
- las dependencias se reinstalan automáticamente.

¿Cómo se recuperan las dependencias?

Con:

npm install

3. Archivo .gitignore

Crear archivo:

.gitignore

Contenido:

node_modules/

.env

Explicación

Línea	Función
node_modules/	Ignorar dependencias
.env	Ocultar datos sensibles



4. ¿Por qué NO subir el .env?

Porque contiene:

- usuarios BD
- contraseñas
- puertos
- configuraciones sensibles

Mala práctica

DB_PASSWORD=123456

Subir eso a GitHub es un riesgo de seguridad.

5. ¿Cómo entregar variables de entorno correctamente?

Solución Profesional

Entregar un archivo:

.env.example

Ejemplo

DB_HOST=localhost

DB_PORT=3306

DB_USER=tu_usuario

DB_PASSWORD=tu_password

DB_NAME=api_usuarios



¿Qué hace .env.example?

Sirve como:

- ✓ plantilla
- ✓ documentación
- ✓ guía de configuración

Sin exponer datos reales.

6. Flujo Correcto para Clonar Proyecto

Paso 1

Clonar repositorio:

```
git clone URL_DEL_REPOSITORIO
```

Paso 2

Instalar dependencias:

```
npm install
```

Paso 3

Crear archivo:

```
.env
```

Usando como guía:



`.env.example`

Paso 4

Ejecutar proyecto:

`node src/app.js`

7. Cómo pedir la generación del repositorio al aprendiz

Puedes redactarlo así:

Instrucción Formal de Entrega

Repositorio Base

El aprendiz deberá crear un repositorio GitHub llamado:

`api-node-mysql-base`

El repositorio debe contener:

- Código fuente funcional
 - Arquitectura por capas
 - CRUD usuarios
 - Login básico
 - README.md
 - Archivo `.env.example`
 - Archivo `.gitignore`
-

Repositorio del Reto



El aprendiz deberá crear un segundo repositorio llamado:

api-node-mysql-reto

Donde implementará:

- Mejoras propias
- Nuevos endpoints
- Validaciones
- Refactorización
- Funcionalidades adicionales

8. Estructura Correcta de Entrega

api-node-mysql-base/

```
|  
├── src/  
├── .env.example  
├── .gitignore  
├── package.json  
└── README.md
```

9. ¿Qué debe tener el README.md?

Obligatorio

Descripción proyecto

Tecnologías usadas

Instalación

Variables entorno

Cómo ejecutar



Endpoints

Ejemplos JSON

Autor

10. Ejemplo README Profesional

API Node MySQL

Tecnologías

- Node.js
- Express
- MySQL

Instalación

npm install

Variables entorno

Crear archivo .env basado en .env.example

Ejecutar

node src/app.js

Endpoints



POST /api/users/register

POST /api/users/login

GET /api/users

DELETE /api/users/:id

11. Conceptos Profesionales que Aprende el Aprendiz

Concepto	Aplicado
GitHub	✓
Control versiones	✓
Seguridad	✓
Variables entorno	✓
Buenas prácticas	✓
Documentación técnica	✓
Entrega profesional	✓
Organización backend	✓

